

EX. NO: 7

IMPLEMENTATION OF RSA

AIM:

To write a C program to implement the RSA encryption algorithm.

ALGORITHM:

STEP-1: Select two co-prime numbers as p and q.

STEP-2: Compute n as the product of p and q.

STEP-3: Compute $(p-1)*(q-1)$ and store it in z.

STEP-4: Select a random prime number e that is less than that of z.

STEP-5: Compute the private key, d as $e * \text{mod-1}(z)$.

STEP-6: The cipher text is computed as $\text{message} * \text{mod } n$. STEP-7: Decryption is done as $\text{cipherdmod } n$.

PROGRAM:

```
#include <stdio.h>
```

```
long long modExp(long long base, long long exp, long long mod)
```

```
{
```

```
    long long result = 1;
```

```
    base = base % mod;
```

```
    while (exp > 0)
```

```
    {
```

```
        if (exp % 2 == 1)
```

```
            result = (result * base) % mod;
```

```
        exp = exp / 2;
```

```
        base = (base * base) % mod;
```

```
    }
```

```
    return result;
}
```

```
int gcd(int a, int b)
{
    while (b != 0)
    {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}
```

```
int modInverse(int e, int phi)
{
    int d;

    for (d = 1; d < phi; d++)
    {
        if ((e * d) % phi == 1)
            return d;
    }

    return -1;
}
```

```
int isPrime(int n)
{
```

```
if (n < 2)
    return 0;
```

```
for (int i = 2; i * i <= n; i++)
{
    if (n % i == 0)
        return 0;
}
```

```
return 1;
}
```

```
int main()
```

```
{
    int p, q;
```

```
    printf("Enter first prime number: ");
    scanf("%d", &p);
```

```
    printf("Enter second prime number: ");
    scanf("%d", &q);
```

```
    if (!isPrime(p) || !isPrime(q) || p == q)
    {
        printf("Invalid prime numbers.\n");
        return 0;
    }
```

```
    int n = p * q;
```

```
int phi = (p - 1) * (q - 1);
```

```
int e;
```

```
for (e = 2; e < phi; e++)
```

```
{
```

```
    if (gcd(e, phi) == 1)
```

```
        break;
```

```
}
```

```
int d = modInverse(e, phi);
```

```
printf("\nPublic Key = (%d, %d)\n", e, n);
```

```
printf("Private Key = (%d, %d)\n", d, n);
```

```
char msg[100];
```

```
printf("\nEnter message (lowercase): ");
```

```
scanf("%s", msg);
```

```
long long cipher[100];
```

```
printf("\nEncrypted Message:\n");
```

```
int i;
```

```
for (i = 0; msg[i] != '\0'; i++)
```

```
{
```

```
    int m = msg[i];
```

```
    cipher[i] = modExp(m, e, n);
    printf("%lld ", cipher[i]);
}

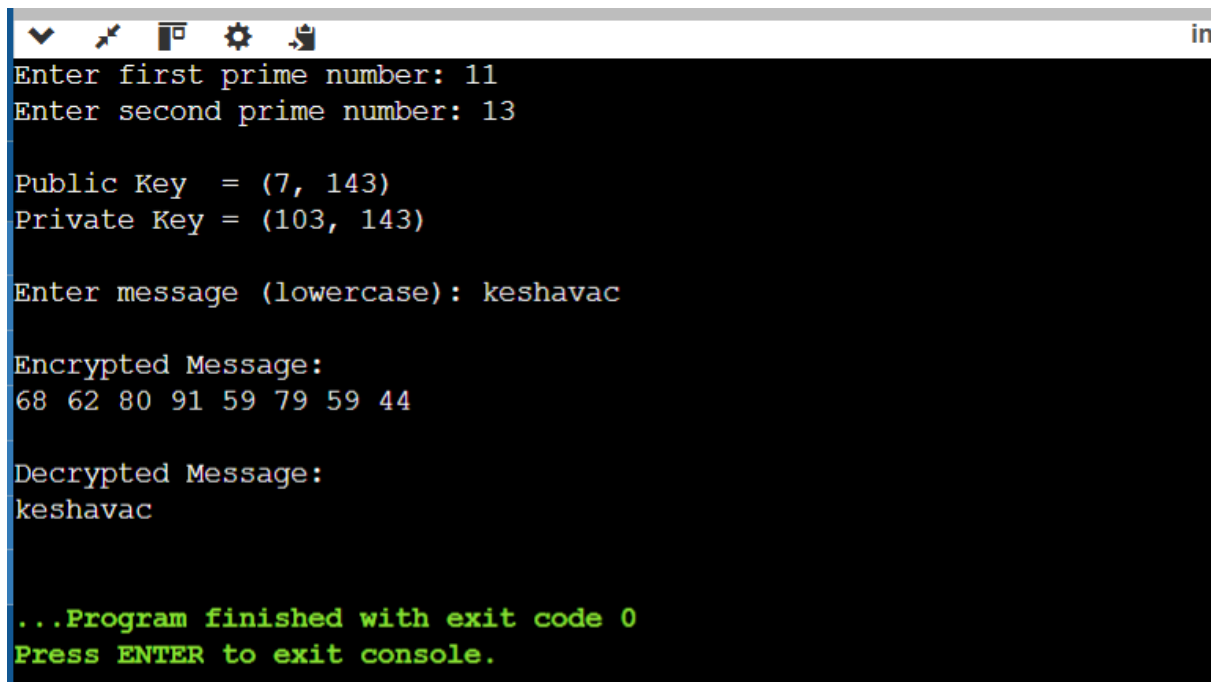
printf("\n\nDecrypted Message:\n");

for (i = 0; msg[i] != '\0'; i++)
{
    char ch = (char)modExp(cipher[i], d, n);
    printf("%c", ch);
}

printf("\n");

return 0;
}
```

OUTPUT:

A screenshot of a terminal window with a dark background and light-colored text. The window has a title bar with standard icons (minimize, maximize, close) and a small 'in' icon on the right. The text inside the terminal shows the execution of an RSA program. It starts with prompts for two prime numbers, 11 and 13. Then it displays the calculated Public Key (7, 143) and Private Key (103, 143). Next, it prompts for a message, 'keshavac', and shows the encrypted message as a sequence of numbers: 68 62 80 91 59 79 59 44. Finally, it shows the decrypted message, 'keshavac', and ends with a green-colored message: '...Program finished with exit code 0' and 'Press ENTER to exit console.'

```
Enter first prime number: 11
Enter second prime number: 13

Public Key = (7, 143)
Private Key = (103, 143)

Enter message (lowercase): keshavac

Encrypted Message:
68 62 80 91 59 79 59 44

Decrypted Message:
keshavac

...Program finished with exit code 0
Press ENTER to exit console.
```

RESULT:

Thus the C program to implement RSA encryption technique had been implemented successfully

